SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Operating Systems
COURSE CODE: CSA-0409

COURSE OUTCOME COVERED

CO3: Evaluate memory management mechanisms such as linking, paging, and segmentation to determine their effectiveness for different system requirements

Assessment Tool Weightage: 40%

QUESTIONS: 5 **Total Marks: 50**

Experiment Questions:

1.Design and implement a program to simulate the translation of a logical address into a physical address using base and limit registers. The program should accept logical address as input, check for validity, and compute the corresponding physical address. Display appropriate messages for valid and invalid addresses.

2.Write a program to demonstrate the paging memory management scheme. The program should divide memory into fixed-size pages and frames, accept page number and offset as input, and compute the corresponding physical address using a page table.

3.Develop a program to simulate a page table. The program should allow the user to input page table entries and logical addresses, then map them to physical addresses. Clearly show how page numbers are translated into frame numbers.

4.Implement a program to simulate the working of a Translation Lookaside Buffer (TLB). The program should demonstrate how TLB improves memory access time by calculating the effective memory access time with and without TLB.

5.Write a program to illustrate memory protection using base and limit registers. The program should verify whether a given logical address lies within the valid range and prevent unauthorized access.

RUBRICS FOR CALCULATION:

Criteria	Weightage (Marks)	Level 4 – Exemplary (Full Marks)	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Conceptual Understanding	35 Marks	Demonstrates clear, complete, and in-depth understanding of OS concepts (types, structure, system calls)	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answers	25 Marks	All answers are correct, precise, and relevant	Mostly correct with minor errors	Partially correct with noticeable errors	Mostly incorrect or irrelevant
Application of Knowledge	15 Marks	Effectively applies concepts to scenarios/questions	Applies with minor mistakes	Limited or partial application	No meaningful application
Completeness of Response	15 Marks	All parts of each question are fully answered	Most parts covered with small omissions	Some parts missing	Incomplete answers
Clarity & Presentation	10 Marks	Clear, well-structured, and neatly presented answers	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

1. Logical Address to Physical Address Using Base and Limit Registers

1. Aim

To simulate the translation of a logical address into a physical address using **Base and Limit Registers**, and identify valid and invalid addresses.

2. Algorithm

1. Read Base Register, Limit Register and Logical Address.
2. Check whether Logical Address < Limit.
3. If valid, calculate the physical address.
4. Otherwise, deny memory access.

Formula

$$\text{Physical Address} = \text{Base Register} + \text{Logical Address}$$

3. Program (C)

```
#include <stdio.h>

int main()
{
   int base, limit, logical, physical;

   printf("Enter Base Register: ");
   scanf("%d", &base);

   printf("Enter Limit Register: ");
   scanf("%d", &limit);

   printf("Enter Logical Address: ");
   scanf("%d", &logical);

   if (logical >= 0 && logical < limit)
   {
      physical = base + logical;

      printf("\nAddress is VALID");
      printf("\nPhysical Address = %d\n", physical);
   }
   else
   {
      printf("\nAddress is INVALID");
      printf("\nAccess Denied\n");
   }
```

```
    return 0;
}
```

5. Sample Calculation

Input:
Base Register = 4000
Limit Register = 1000
Logical Address = 350
Since,

$$350<1000$$

Address is **valid**.

$$\text{Physical Address}=4000+350=4350$$

6. Sample Output

```
Enter Base Register: 4000
Enter Limit Register: 1000
Enter Logical Address: 350

Address is VALID
Physical Address = 4350
```

Result:

The program successfully translates a valid logical address into its corresponding physical address and denies invalid memory access.

2. Paging Memory Management

1.Aim

To simulate **paging memory management** by accepting a page number and offset and calculating the corresponding physical address using a page table.

2.Algorithm and Formula

1. Read the **page number** and **offset**.
2. Find the corresponding **frame number** from the page table.
3. Calculate the physical address.

$$\text{Physical Address} = (\text{Frame Number} \times \text{Page Size}) + \text{Offset}$$

3.Program (C)

```
#include <stdio.h>

int main()
{
   int page, offset, frame, page_size, physical;

   printf("Enter Page Size: ");
   scanf("%d", &page_size);

   printf("Enter Page Number: ");
   scanf("%d", &page);

   printf("Enter Frame Number: ");
   scanf("%d", &frame);

   printf("Enter Offset: ");
   scanf("%d", &offset);

   if (offset >= 0 && offset < page_size)
   {
      physical = (frame * page_size) + offset;

      printf("\nPage Number     : %d", page);
      printf("\nFrame Number    : %d", frame);
      printf("\nPhysical Address: %d\n", physical);
   }
   else
   {
      printf("\nInvalid Offset\n");
```

```
    }

    return 0;
}
```

4. **Sample Calculation**

Input:
Page Size = 100
Page Number = 2
Frame Number = 5
Offset = 30

$$\text{Physical Address} = (5 \times 100) + 30$$
$$= 530$$

5. **Sample Output**

```
Enter Page Size: 100
Enter Page Number: 2
Enter Frame Number: 5
Enter Offset: 30
Page Number     : 2
Frame Number    : 5
Physical Address: 530
```

Result:
The program successfully performs paging address translation and calculates the physical address from the frame number and offset.

3. Page Table Simulation

1. Aim

To simulate a **page table** that maps a logical **page number to a physical frame number** and calculates the corresponding physical address.

2. Algorithm and Formula

1. Read the page-table entries.
2. Read the page number and offset.
3. Find the corresponding frame number.
4. Calculate the physical address.

$$\text{Physical Address} = (\text{Frame Number} \times \text{Page Size}) + \text{Offset}$$

3. Program (C)

```
#include <stdio.h>

int main()
{
   int pageTable[10], n;
   int page, offset, pageSize, frame, physical;
   int i;

   printf("Enter Page Size: ");
   scanf("%d", &pageSize);

   printf("Enter Number of Pages: ");
   scanf("%d", &n);

   printf("Enter Page Table (Page -> Frame):\n");

   for (i = 0; i < n; i++)
   {
      printf("Page %d -> Frame: ", i);
      scanf("%d", &pageTable[i]);
   }

   printf("Enter Page Number: ");
   scanf("%d", &page);
```

```
    printf("Enter Offset: ");
    scanf("%d", &offset);

    if (page >= 0 && page < n && offset >= 0 && offset < pageSize)
    {
        frame = pageTable[page];
        physical = (frame * pageSize) + offset;

        printf("\nPage Number     : %d", page);
        printf("\nFrame Number    : %d", frame);
        printf("\nPhysical Address: %d\n", physical);
    }
    else
    {
        printf("\nInvalid Page Number or Offset\n");
    }

    return 0;
}
```

4. Sample Calculation

Page Table:
Page 0 -> Frame 5
Page 1 -> Frame 8
Page 2 -> Frame 1
Page 3 -> Frame 6
Input:
Page Size = 100
Page Number = 2
Offset = 30
From the page table:

$$\text{Page } 2 \rightarrow \text{Frame } 1$$

Therefore,

$$\text{Physical Address} = (1 \times 100) + 30 = 130$$

5. Sample Output

```
Page Number     : 2
Frame Number    : 1
Physical Address: 130
```

Result:
The program successfully performs **page-to-frame mapping** using a page table and calculates the corresponding physical address.

4.Translation Lookaside Buffer (TLB) Simulation

1. **Aim**

To simulate a **Translation Lookaside Buffer (TLB)** and calculate the **Effective Memory Access Time (EMAT)** with and without TLB.

2. **Algorithm and Formula**

1. Read TLB access time and main memory access time.
2. Read the TLB hit ratio.
3. Calculate EMAT with TLB.
4. Calculate memory access time without TLB.
5. Compare both values.

With TLB

$$EMAT = h(T+M) + (1-h)(T+2M)$$

Where:

- = TLB hit ratio
- = TLB access time
- = Memory access time

Without TLB

$$\text{Memory Access Time} = 2M$$

3. **Program (C)**

```
#include <stdio.h>

int main()
{
   float hit, tlb, memory;
   float emat, without_tlb;

   printf("Enter TLB Access Time (ns): ");
   scanf("%f", &tlb);

   printf("Enter Memory Access Time (ns): ");
```

```
    scanf("%f", &memory);

    printf("Enter TLB Hit Ratio (0 to 1): ");
    scanf("%f", &hit);

    emat = hit * (tlb + memory)
        + (1 - hit) * (tlb + 2 * memory);

    without_tlb = 2 * memory;

    printf("\nEMAT with TLB    = %.2f ns", emat);
    printf("\nAccess Time without TLB = %.2f ns\n", without_tlb);

    return 0;
}
```

4. Sample Calculation

Given:
TLB Access Time = 10 ns
Memory Access Time = 100 ns
TLB Hit Ratio = 0.90

With TLB

$$EMAT=0.9(10+100)+0.1(10+200)$$
$$=99+21$$
$$EMAT=120 \text{ ns}$$

Without TLB

$$=2(100)$$
$$=200 \text{ ns}$$

5. Sample Output

```
Enter TLB Access Time (ns): 10
Enter Memory Access Time (ns): 100
Enter TLB Hit Ratio (0 to 1): 0.90

EMAT with TLB    = 120.00 ns
Access Time without TLB = 200.00 ns
```

Result:
The program successfully demonstrates that the **TLB reduces effective memory access time** compared with memory access without TLB.

5.Memory Protection Using Base and Limit Registers

1. Aim

To implement **memory protection** using Base and Limit Registers and verify whether a logical address is within the permitted memory range.

2. Algorithm and Formula

1. Read the **Base Register**, **Limit Register**, and **Logical Address**.
2. Check whether the logical address is within the valid range.
3. If valid, calculate the physical address.
4. If invalid, deny memory access.

$$\text{Valid if } 0 \leq \text{Logical Address} < \text{Limit}$$
$$\text{Physical Address} = \text{Base Register} + \text{Logical Address}$$

3. Program (C)

```
#include <stdio.h>

int main()
{
   int base, limit, logical, physical;

   printf("Enter Base Register: ");
   scanf("%d", &base);

   printf("Enter Limit Register: ");
   scanf("%d", &limit);

   printf("Enter Logical Address: ");
   scanf("%d", &logical);

   if (logical >= 0 && logical < limit)
   {
      physical = base + logical;
```

```
        printf("\nAccess: ALLOWED");
        printf("\nPhysical Address = %d\n", physical);
    }
    else
    {
        printf("\nAccess: DENIED");
        printf("\nInvalid Logical Address\n");
    }

    return 0;
}
```

4. Sample Calculation

Given:
Base Register = 5000
Limit Register = 1000
Logical Address = 600
Since,
Access is **allowed**.
For an invalid address:
Logical Address = 1200
Since,

$$600<1000$$

Access is **allowed**.

$$\text{Physical Address}=5000+600$$
$$=5600$$

For an invalid address:
Logical Address = 1200
Since,
1200>1000
Access = Denied

5. Sample Output

Enter Base Register: 5000
Enter Limit Register: 1000
Enter Logical Address: 600

Access: ALLOWED
Physical Address = 5600

Result:
The program successfully checks memory access using Base and Limit Registers and prevents access to an invalid logical address.